

**LECTURA Y TRAZABILIDAD
DEL CÓDIGO EN JAVA**



Carlos Andrés Tamayo Benjumea

**Universidad Popular del Cesar – Seccional Aguachica
Facultad de Ingenierías y Tecnologías**

Índice

1. Introducción	2
2. Representación visual	3
3. Código a analizar	4
4. Primera lectura del código	4
5. Tabla de seguimiento del estado	5
6. ¿Qué debemos registrar?	6
7. Preguntas de reflexión	7
8. Verificación	8
9. Actividad 2 – Búsqueda en un arreglo de String	9

1. Introducción

En programación no basta con saber escribir código. Un buen programador también debe ser capaz de **leer, comprender y predecir el comportamiento de un programa** escrito por otra persona.

En esta actividad utilizaremos el **método de ordenamiento de burbuja (Bubble Sort)** como medio para desarrollar esta habilidad.

El objetivo principal de la actividad **no es memorizar el algoritmo**, sino aprender a seguir la ejecución de un programa paso a paso, observando cómo cambian las variables y cómo se modifica el contenido de un arreglo.

Durante la actividad analizaremos:

- Los valores de las variables.
- Los valores de los índices i y j .
- Los elementos del arreglo que se están comparando.
- La condición del bloque `if`.
- Cuándo ocurre un intercambio.
- Cómo cambia el arreglo después de cada operación.
- El estado final del programa.

Regla de la actividad

Antes de ejecutar el programa, intentaremos determinar qué hará.
La secuencia de trabajo será:

Leer → Predecir → Registrar → Verificar

2. Representación visual

Antes de analizar el código, observa la siguiente representación del proceso.

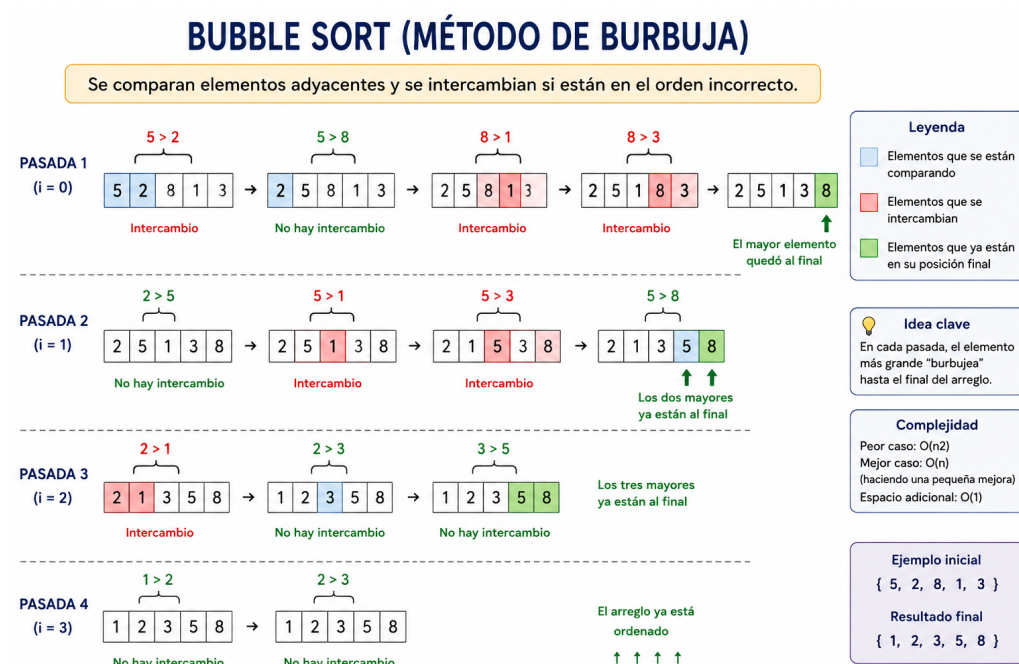


Figura 1: Representación visual del método de burbuja.

Observa cómo los elementos vecinos son comparados y, cuando es necesario, intercambian sus posiciones.

3. Código a analizar

A continuación se presenta el código Java que debemos analizar.

```
1 public class Main {
2
3     public static void main(String[] args) {
4
5         int[] numeros = {5, 2, 8, 1, 3};
6
7         for (int i = 0; i < numeros.length - 1; i++) {
8
9             for (int j = 0; j < numeros.length - 1 - i; j++) {
10
11                 if (numeros[j] > numeros[j + 1]) {
12
13                     int temporal = numeros[j];
14                     numeros[j] = numeros[j + 1];
15                     numeros[j + 1] = temporal;
16                 }
17             }
18         }
19     }
20 }
```

Importante: inicialmente no ejecutes el programa. Primero intenta determinar manualmente qué ocurrirá.

4. Primera lectura del código

Antes de comenzar la trazabilidad, identifica las siguientes partes:

Elemento	¿Qué crees que hace?
<code>int[] numeros</code>	
<code>numeros.length</code>	
Primer for	
Segundo for	
if	
Variable temporal	

5. Tabla de seguimiento del estado

La siguiente tabla permitirá registrar el estado del programa durante su ejecución.

En cada paso debemos observar qué valores tienen las variables, qué elementos se comparan, si ocurre un intercambio y cómo queda el arreglo.

Paso	<i>i</i>	<i>j</i>	<code>numeros[j]</code>	<code>numeros[j+1]</code>	Comparación	Intercambia	Estado del arreglo
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							
14							
15							

6. ¿Qué debemos registrar?

- **Paso:** número que identifica cada comparación realizada.
- i : valor actual de la variable correspondiente al ciclo externo.
- j : valor actual de la variable correspondiente al ciclo interno.
- `numeros[j]`: elemento actual que estamos analizando.
- `numeros[j+1]`: elemento que se encuentra inmediatamente después.
- **Comparación:** condición que está evaluando el programa.
- **Intercambia:** indicar **Sí** o **No**.
- **Estado del arreglo:** escribir cómo queda el arreglo después de la operación.

Por ejemplo, si:

$$j = 0$$

entonces:

$$\text{numeros}[j] = \text{numeros}[0] = 5$$

y:

$$\text{numeros}[j+1] = \text{numeros}[1] = 2$$

La comparación sería:

$$5 > 2$$

Como la condición es verdadera, se produce un intercambio.

7. Preguntas de reflexión

Una vez completada la tabla, responde las siguientes preguntas.

Pregunta 1

¿Qué comportamiento observas en los elementos del arreglo a medida que avanza el programa?

Pregunta 2

¿Qué elemento parece desplazarse hacia el final después de cada recorrido?

Pregunta 3

¿Por qué se utiliza la variable `temporal`?

Pregunta 4

¿Qué crees que hace finalmente el programa?

8. Verificación

Después de completar la tabla y responder las preguntas:

1. Ejecuta el programa.
2. Observa el resultado.
3. Compara el resultado con tu predicción.
4. Identifica los errores que hayas cometido en la trazabilidad.

Conclusión

El objetivo de esta actividad no es solamente descubrir qué hace el método de burbuja.

El objetivo es desarrollar la capacidad de **leer código, seguir su estado y predecir su comportamiento antes de ejecutarlo.**

9. Actividad 2 – Búsqueda en un arreglo de String

En esta actividad analizaremos un programa que trabaja con un arreglo de cadenas de texto. El programa solicita al usuario un carácter y posteriormente recorre el arreglo para determinar qué palabras comienzan con dicho carácter.

Leer → Predecir → Registrar → Verificar

Antes de ejecutar

No ejecutes el programa inicialmente.

Primero lee cuidadosamente el código y realiza la trazabilidad. Intenta predecir qué ocurrirá cuando el usuario ingrese un carácter.

Código a analizar

```
1 import java.util.Scanner;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Scanner entrada = new Scanner(System.in);
8
9         String[] palabras = {
10             "Java",
11             "Python",
12             "JavaScript",
13             "Csharp",
14             "Javascript",
15             "Kotlin"
16         };
17
18         System.out.print("Ingresa un caracter: ");
19         char caracter = entrada.next().charAt(0);
20
21         System.out.println("\nPalabras encontradas:");
22
23         for (int i = 0; i < palabras.length; i++) {
24
25             if (palabras[i].charAt(0) == caracter) {
26
27                 System.out.println(
28                     i + " -> " + palabras[i]
29                 );
30             }
31         }
32     }
33 }
```

Entrada para el análisis

Para realizar la trazabilidad, suponga que el usuario ingresa:

J

Antes de ejecutar el programa, predice qué palabras serán mostradas.

Primera lectura del código

Identifica qué función cumple cada elemento del programa.

Elemento	¿Qué crees que hace?
<code>String[] palabras</code>	
<code>palabras.length</code>	
<code>char character</code>	
<code>charAt(0)</code>	
<code>for</code>	
<code>if</code>	
<code>palabras[i]</code>	

Tabla de seguimiento del estado

Realiza la ejecución manual del programa y registra el estado de las variables en cada iteración.

Paso	<i>i</i>	palabras[i]	charAt(0)	caracter	¿Son iguales?	¿Qué ocurre?
1						
2						
3						
4						
5						
6						

Predicción del resultado

Antes de ejecutar el programa, escribe qué crees que aparecerá en pantalla.

Mi predicción:

Verificación

Ahora ejecuta el programa y compara el resultado obtenido con tu predicción.

1. ¿Tu predicción fue correcta?
2. Si fue incorrecta, ¿en qué paso de la trazabilidad cometiste el error?
3. ¿Qué valor toma la variable *i* en cada iteración?
4. ¿Qué valor produce `palabras[i].charAt(0)`?
5. ¿Qué condición debe cumplirse para que se ejecute el `println`?

Reto: modificar la entrada

Sin modificar el código, realiza nuevamente la trazabilidad para cada uno de los siguientes caracteres:

Caracter	¿Qué palabras se mostrarán?	¿Por qué?
P		
C		
K		
Z		

Reflexión

Responde con tus propias palabras:

1. ¿Qué representa la variable `i`?
2. ¿Qué representa `palabras[i]`?
3. ¿Qué hace `charAt(0)`?
4. ¿Por qué se utiliza `charAt(0)` después de leer el dato mediante `next()`?
5. ¿Qué ocurriría si el usuario ingresara una letra que no aparece como primera letra de ninguna palabra?
6. ¿Qué tendríamos que modificar para que la búsqueda no distinga entre mayúsculas y minúsculas?

Idea clave

El objetivo de esta actividad no es solamente obtener el resultado correcto. El objetivo es aprender a seguir el estado del programa y comprender por qué cada instrucción produce un determinado resultado.

Leer el código antes de ejecutarlo.